

W15D01 — Structured Logging & Observability

ML Engineer Journey | Phase 4 — Database & Observability | 15 Juni 2026

1. Kenapa print() Berbahaya di Production

Bayangkan kamu adalah manajer sebuah pabrik besar. Setiap mesin bekerja, tapi tidak ada satu papan monitor pun — tidak ada lampu indikator, tidak ada alarm. Suatu hari ada yang salah, produk cacat keluar dari jalur. Tapi kamu tidak tahu **kapan** itu mulai terjadi, **di mesin mana**, atau **apa penyebabnya** — karena tidak ada yang mencatat.

Itulah kondisi aplikasi yang hanya mengandalkan **print()**. `print()` seperti pekerja yang berteriak sesuatu di tengah lantai pabrik — sekali terdengar, lalu hilang. Tidak ada catatan, tidak ada timestamp, tidak bisa difilter, tidak bisa dimatikan.

Python **logging module** adalah sistem papan monitor pabrik itu. Setiap event dicatat dengan waktu, tingkat kepentingan, sumber, dan pesan. Kamu bisa atur mana yang ditampilkan, mana yang disimpan ke file, mana yang dikirim ke alerting.

Tabel Perbandingan: `print()` vs logging

Aspek	<code>print()</code>	logging
Timestamp	Tidak ada	Otomatis
Level / Severity	Tidak ada	DEBUG / INFO / WARNING / ERROR / CRITICAL
Bisa dimatikan	Hanya hapus kode manual	Atur level threshold
Output destination	Hanya stdout	Console, file, network, cloud
Bisa difilter	Tidak	Per module, per level
Format	Bebas, tidak konsisten	Terstruktur, bisa JSON
Source / module info	Tidak ada	<code>%(name)s</code> otomatis
Production-safe	Berbahaya	Dirancang untuk production

2. Arsitektur Python Logging Module

Logging module bekerja dengan empat komponen utama yang saling terhubung:

- **Logger** — objek yang kamu panggil untuk menulis log. Dibuat per module dengan `getLogger(__name__)`.
- **Handler** — menentukan ke mana log dikirim (`StreamHandler` ke console, `FileHandler` ke file, dll.).

- **Formatter** — menentukan format visual output (timestamp, level, nama module, pesan).
- **Level** — threshold kepentingan. Hanya log dengan level $\geq$ threshold yang diproses.

Hubungan antar komponen: Logger menerima panggilan log dari kode, mengecek apakah level memenuhi threshold, lalu meneruskan ke satu atau lebih Handler. Tiap Handler memformat output melalui Formatter-nya sebelum mengirim ke tujuan akhir.

Tabel Level Logging

Level	Nilai	Kapan Dipakai	Analogi Dokter IGD
DEBUG	10	Detail teknis internal, saat debugging	Catatan nadi dan tekanan darah per menit
INFO	20	Alur normal aplikasi berjalan	Catatan rekam medis: pasien masuk, keluar
WARNING	30	Kondisi tidak ideal tapi masih bisa di-handle	Tanda kuning: pasien alergi penisilin
ERROR	40	Kejadian buruk yang sudah di-handle	Reaksi alergi terjadi, sudah diberi epinefrin
CRITICAL	50	Sistem kolaps, perlu tindakan segera	ICU penuh, generator mati

3. Implementasi: logging_config.py

Prinsip utama: logging adalah infrastruktur — dikonfigurasi **sekali** di entry point (main.py), lalu seluruh module tinggal memanggil getLogger(). Ini adalah penerapan langsung Separation of Concerns dari W11 — router/service/repository tidak punya urusan dengan cara log dikonfigurasi.

app/core/logging_config.py

```
import logging
import sys

def setup_global_logging(level: str = 'INFO') -> None:
    # Root logger – semua logger anak inherit dari sini
    root_logger = logging.getLogger()

    # Guard: cegah duplicate handler jika dipanggil lebih dari sekali
    if root_logger.hasHandlers():
        return

    # Tangkap semua level di logger (filter di handler)
    root_logger.setLevel(logging.DEBUG)

    # Handler ke stdout
    handler = logging.StreamHandler(sys.stdout)
    handler.setLevel(getattr(logging, level.upper(), logging.INFO))

    # Format: [timestamp] [LEVEL] [module:line] - message
    formatter = logging.Formatter(
        fmt='[%asctime)s] [%levelname)s] [%name)s:%(lineno)d] - %(message)s',
        datefmt='%Y-%m-%d %H:%M:%S'
    )
```

```

handler.setFormatter(formatter)
root_logger.addHandler(handler)

# Matikan noise dari library external
logging.getLogger('uvicorn.access').setLevel(logging.WARNING)
logging.getLogger('sqlalchemy.engine').setLevel(logging.WARNING)

```

Penjelasan Baris Kritis

- **getLogger()** tanpa argumen → root logger. Semua logger lain adalah anaknya dan inherit konfigurasi ini.
- **hasHandlers() guard** → mencegah handler terganggu jika setup dipanggil dua kali (misalnya saat testing dengan auto-reload).
- **root_logger.setLevel(DEBUG)** → tangkap semua. Filter sesungguhnya ada di handler — ini memberi fleksibilitas menambah handler dengan level berbeda nanti.
- **%(name)s:%(lineno)d** → nama module dan nomor baris — kombinasi ini yang membuat log mudah di-trace saat debugging production.
- **uvicorn.access** dan **sqlalchemy.engine** → library ini sangat verbose; dimatikan di INFO agar log kamu tidak tenggelam di noise mereka.

4. Cara Pakai di Setiap Layer

Satu aturan sederhana: setiap file Python yang butuh logging cukup tulis dua baris di bagian atas — import logging dan deklarasi logger. Tidak lebih.

main.py — Setup sekali, di lifespan

```

import logging
from app.core.logging_config import setup_global_logging

# Panggil PERTAMA sebelum apapun diinisialisasi
setup_global_logging()

# Logger untuk file ini — __name__ = 'main' atau '__main__'
logger = logging.getLogger(__name__)

@asynccontextmanager
async def lifespan(app: FastAPI):
    logger.info('Application starting up')
    yield
    logger.info('Application shutting down')

```

Service / Router / Repository — Tinggal getLogger(__name__)

```

# Di FILE MANAPUN — tidak perlu import setup_global_logging
import logging

logger = logging.getLogger(__name__)

# __name__ otomatis = nama module, misal: 'app.services.analysis_services'

# Penggunaan:

```

<code>logger.info('Prediction requested')</code>	<code># alur normal</code>
<code>logger.debug(f'Raw output: {result}')</code>	<code># detail teknis</code>
<code>logger.warning('Missing optional field')</code>	<code># tidak ideal tapi OK</code>
<code>logger.error('Failed', exc_info=True)</code>	<code># error + full traceback</code>
<code>logger.critical('System failure detected')</code>	<code># darurat</code>

5. Parameter Kritis: `exc_info=True`

Ini adalah parameter yang paling sering dilupakan dan paling mahal konsekuensinya saat tidak ada.

Tanpa `exc_info=True`

```
try:
    result = model.predict(data)
except Exception as e:
    logger.error(f'Prediction failed: {str(e)}')
# Output: [2026-06-15 10:23:01] [ERROR] [...] - Prediction failed: list index out
of range
# Kamu tahu APA yang gagal, tapi tidak tahu DI MANA dan LEWAT JALUR MANA.
```

Dengan `exc_info=True`

```
try:
    result = model.predict(data)
except Exception as e:
    logger.error(f'Prediction failed: {str(e)}', exc_info=True)
# Output:
# [2026-06-15 10:23:01] [ERROR] [...] - Prediction failed: list index out of rang
e
# Traceback (most recent call last):
#   File 'app/services/analysis_services.py', line 45, in predict
#     result = model.predict(data)
#   File 'app/ml/model.py', line 123, in predict
#     return self.weights[index]
# IndexError: list index out of range
```

`exc_info=True` memerintahkan logging module untuk menangkap seluruh exception context yang sedang aktif — traceback lengkap, call stack, nama file, nomor baris. Di production, ini satu-satunya cara kamu tahu baris mana yang gagal tanpa harus mereproduksi bug secara manual.

6. Kenapa Selalu `__name__`, Bukan Hardcode String

Python logging dirancang dengan hierarki pohon. Saat kamu memanggil `getLogger('app.services.analysis_services')`, logger itu otomatis menjadi anak dari `'app.services'`, yang menjadi anak dari `'app'`, yang menjadi anak dari root logger. Semua konfigurasi handler dan formatter dipropagasi dari atas ke bawah.

```
# Opsi A – SALAH (hardcode)
logger = logging.getLogger('app.services.analysis')
# Jika file dipindahkan atau di-rename: nama logger SALAH secara diam-diam.
# Kamu tidak akan dapat error – log hanya akan salah namespace.

# Opsi B – BENAR (__name__)
logger = logging.getLogger(__name__)
# Python menghitung nilainya dari lokasi file secara absolut.
# Rename file → nama logger otomatis ikut berubah.
# Zero maintenance, zero typo risk.
```

7. Pattern yang Diimplementasikan di Project

Berdasarkan kode yang sudah direview dan diperbaiki, ini adalah pattern final yang digunakan di DSS API (Data Science System):

Struktur File

```
app/
├── core/
│   ├── logging_config.py ← setup_global_logging() – satu-satunya tempat konfigurasi
│   ├── main.py           ← import dan panggil setup_global_logging() satu kali
│   └── routers/
│       ├── predictions.py ← logger = logging.getLogger(__name__) saja
│       └── services/
│           └── analysis_services.py ← logger = logging.getLogger(__name__) saja
```

Contoh Output Terminal yang Benar

```
[2026-06-15 12:35:09] [INFO] [main:27] - Memulai Data Science System API...
[2026-06-15 12:35:28] [INFO] [app.routers.predictions:35] - Menerima request analisis baru: 'Prediksi Harga Rumah'
[2026-06-15 12:35:28] [INFO] [app.services.analysis_services:66] - Memulai komputasi ML...
[2026-06-15 12:35:43] [INFO] [app.services.analysis_services:88] - Selesai komputasi ML...
```

8. Ringkasan Poin Penting

- **setup_global_logging()** dipanggil **SEKALI** di main.py — tidak di router, service, atau repository.
- **getLogger(__name__)** di **SEMUA** file — tidak ada hardcode string, tidak ada pengecualian.
- **exc_info=True wajib** di setiap `logger.error()` dan `logger.critical()` di dalam except block.
- **Guard hasHandlers()** mencegah handler terganggu saat testing atau hot-reload.

- **Level di root logger = DEBUG**, filter sesungguhnya ada di handler (INFO default production).
- **Matikan noise library external**: `uvicorn.access` dan `sqlalchemy.engine` diset ke WARNING.
- **Logging adalah infrastruktur** — sama seperti database engine atau config settings, dikonfigurasi di satu tempat.

Hubungan dengan materi sebelumnya: Separation of Concerns (W11) berlaku juga di logging. Router tidak mengkonfigurasi database, service tidak mengkonfigurasi router — dan tidak ada yang boleh mengkonfigurasi logging kecuali entry point.